Programación Lineal Aplicada

Optimización de recursos en organizaciones

Juan Ignacio da Torre

Laboratorio de Métodos Cuantitativos Aplicados a la Gestión
Universidad de Buenos Aires

Primer cuatrimestre 2026

Objetivo: Resolver problemas reales de asignación óptima de recursos usando programación lineal

Objetivo de la charla

¿Qué vamos a ver hoy?

- Entender problemas organizacionales como sistemas de ecuaciones.
- Fundamentos de programación lineal (PL).
- Formular modelos de optimización para problemas reales.
- Resolver con Python: `linprog` y `PuLP`.
- Analizar casos prácticos: producción, logística, personal.

¿Por qué Optimizar?

De decisiones intuitivas a decisiones óptimas

La optimización en las organizaciones

Problemas cotidianos:

- ¿Cuánto producir de cada producto?
- ¿Cómo distribuir la mercadería?
- ¿Cuántos empleados contratar?
- ¿Cómo asignar turnos?
- ¿Qué mezcla minimiza costos?

Común denominador

Todos buscan:

- Maximizar ganancias
- Minimizar costos
- Cumplir restricciones
- Usar recursos limitados

"Los recursos son limitados, las necesidades son infinitas"

Modelado mediante sistemas de ecuaciones

Caso simple: Una panadería produce pan y medialunas.

Recursos limitados:

- 160 horas de horno
- 90 horas de mano de obra

Consumo por unidad:

- Pan: 1h horno + 0.5h trabajo
- Medialuna: 2h horno + 1h trabajo

Variables de decisión:

- x_1 = unidades de pan a producir
- x_2 = unidades de medialunas a producir

Sistema de ecuaciones:

$$\begin{cases} x_1 + 2x_2 \leq 160 \text{ (horno)} \\ 0.5x_1 + x_2 \leq 90 \text{ (trabajo)} \\ x_1, x_2 \geq 0 \end{cases}$$

Objetivo:

$$\text{máx } Z = 50x_1 + 80x_2$$

Este es un problema de programación lineal

Programación Lineal

Fundamentos matemáticos

¿Qué es la Programación Lineal?

Definición

Técnica matemática para asignar recursos limitados entre actividades competitivas de manera óptima.

Componentes esenciales:

1. **Variables de decisión:** Lo que podemos controlar $(x_1, x_2, \ldots, x_n)$
2. **Función objetivo:** Lo que queremos maximizar o minimizar

$$\text{máx} / \text{mín} \quad Z = c_1x_1 + c_2x_2 + \cdots + c_nx_n$$

3. **Restricciones:** Limitaciones y condiciones

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \leq b_1$$

4. **No negatividad:** $x_i \geq 0$ (típicamente)

Forma estándar de un problema de PL

Problema general:

$$\begin{aligned} \text{Maximizar/Minimizar} \quad & Z = \mathbf{c}^T\mathbf{x} \\ \text{Sujeto a:} \quad & \mathbf{A}\mathbf{x} \leq \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \end{aligned}$$

Donde:

- $\mathbf{x} = (x_1, x_2, \ldots, x_n)^T$: vector de variables
- $\mathbf{c} = (c_1, c_2, \ldots, c_n)^T$: coeficientes de la función objetivo
- $\mathbf{A}$: matriz de coeficientes de las restricciones $(m \times n)$
- $\mathbf{b} = (b_1, b_2, \ldots, b_m)^T$: términos independientes

Linealidad

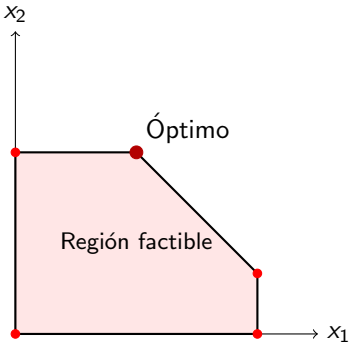
Todas las relaciones son **lineales**: no hay exponentes, productos entre variables, logaritmos, etc.

Región factible y solución óptima

Conceptos clave:

- **Solución factible:** Cualquier solución que cumple todas las restricciones
- **Región factible:** Conjunto de todas las soluciones factibles
- **Solución óptima:** Solución factible que optimiza la función objetivo

Propiedad fundamental: La solución óptima siempre está en un vértice de la región factible.



Tipos de problemas de PL

1. **Producción:** ¿Cuánto producir de cada producto?
   - Maximizar ganancia con recursos limitados
2. **Mezcla:** ¿Qué proporciones usar?
   - Minimizar costo cumpliendo especificaciones
3. **Transporte:** ¿Cómo distribuir desde orígenes a destinos?
   - Minimizar costo de envío
4. **Asignación:** ¿Quién hace qué?
   - Maximizar eficiencia en asignación de tareas
5. **Dieta:** ¿Qué alimentos incluir?
   - Minimizar costo cumpliendo nutrientes mínimos

Formulación de Modelos

Del problema real al modelo matemático

Pasos para formular un modelo de PL

1. **Identificar las variables de decisión**
   - ¿Qué cantidades necesitamos determinar?
   - Usar símbolos claros: x_1, x_2, etc.
2. **Definir la función objetivo**
   - ¿Qué queremos maximizar o minimizar?
   - Expresarla como combinación lineal de las variables
3. **Establecer las restricciones**
   - ¿Qué limitaciones tenemos?
   - Recursos, demandas, capacidades, etc.
4. **Agregar restricciones de no negatividad**
   - Típicamente $x_i \geq 0$
5. **Verificar linealidad**
   - Todas las ecuaciones deben ser lineales

Ejemplo: Producción de una fábrica

Situación: Una fábrica produce dos productos (x_1 y x_2).

Datos:

- Máquina A: 100 horas disponibles
  - x_1: 2 horas
  - x_2: 1 hora
- Máquina B: 80 horas disponibles
  - x_1: 1 hora
  - x_2: 1 hora
- Ganancia:
  - x_1: \$1
  - x_2: \$1

Modelo:

$$\begin{aligned} \text{máx } Z &= x_1 + x_2 \\ \text{s.a.:} \quad 2x_1 + x_2 &\leq 100 \\ x_1 + x_2 &\leq 80 \\ x_1, x_2 &\geq 0 \end{aligned}$$

Variables:

- x_1: unidades de producto 1
- x_2: unidades de producto 2

Resolución con Python

Herramientas: linprog y PuLP

Opción 1: linprog (SciPy)

Características

- Parte de SciPy, librería científica estándar
- Siempre **minimiza**
- Sintaxis algebraica (matrices y vectores)

Sintaxis básica

```
from scipy.optimize import linprog

# Coeficientes (minimizaci n)
c = [c1, c2, ...]

# Restricciones: A_ub @ x <= b_ub
A_ub = [[a11, a12, ...],
        [a21, a22, ...]]
b_ub = [b1, b2, ...]

# Resolver
res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=(0, None))
```

Ejemplo con linprog

Problema: Maximizar $Z = x_1 + x_2$ sujeto a:

$$\begin{cases} 2x_1 + x_2 \leq 100 \\ x_1 + x_2 \leq 80 \\ x_1, x_2 \geq 0 \end{cases}$$

Código

```
from scipy.optimize import linprog

# linprog minimiza, entonces invertimos signos
c = [-1, -1]

# Restricciones
A = [[2, 1],
     [1, 1]]
b = [100, 80]

# Resolver
res = linprog(c, A_ub=A, b_ub=b, bounds=(0, None))

print(f"x1 = {res.x[0]:.2f}")
print(f"x2 = {res.x[1]:.2f}")
print(f"Z m ximo = {-res.fun:.2f}")
```

Opción 2: PuLP

Características

- Librería especializada en optimización
- Sintaxis declarativa (más legible)
- Permite `LpMaximize` o `LpMinimize` explícitamente

Sintaxis básica

```
from pulp import *

# Crear problema
modelo = LpProblem("Nombre", LpMaximize)

# Definir variables
x1 = LpVariable("x1", lowBound=0)
x2 = LpVariable("x2", lowBound=0)

# Funci n objetivo
modelo += c1*x1 + c2*x2
```

Ejemplo con PuLP

Mismo problema: Maximizar $Z = x_1 + x_2$

Código

```
from pulp import *

# Crear modelo
modelo = LpProblem("Produccion", LpMaximize)

# Variables
x1 = LpVariable("x1", lowBound=0)
x2 = LpVariable("x2", lowBound=0)

# Funci n objetivo
modelo += x1 + x2, "Ganancia_Total"

# Restricciones
modelo += 2*x1 + x2 <= 100, "Maquina_A"
modelo += x1 + x2 <= 80, "Maquina_B"

# Resolver
modelo.solve()

# Resultados
print(f"x1 = {x1.varValue:.2f}")
print(f"x2 = {x2.varValue:.2f}")
print(f"Z m ximo = {value(modelo.objective):.2f}")
```

Comparación: linprog vs PuLP

Aspecto	linprog	PuLP
Sintaxis	Algebraica	Declarativa
Objetivo	Solo minimiza	Max o Min
Legibilidad	Menos intuitiva	Más clara
Instalación	Ya en SciPy	Requiere pip
Restricciones	Matrices	Ecuaciones
Resultados	`res.x`, `res.fun`	`.varValue`

Recomendación

- **linprog:** Para problemas simples, uso rápido
- **PuLP:** Para modelos complejos, más legible para compartir código

Casos Prácticos

Aplicaciones en organizaciones reales

Caso 1: Panadería

Contexto: Produce pan y medialunas.

Datos:

- **Pan:**
  - 1h horno, 0.5h trabajo
  - Ganancia: \$50
- **Medialuna:**
  - 2h horno, 1h trabajo
  - Ganancia: \$80
- **Disponibilidad:**
  - 160h horno
  - 90h trabajo

Modelo:

$$\begin{aligned} \text{máx } Z &= 50x_1 + 80x_2 \\ \text{s.a.:} \quad x_1 + 2x_2 &\leq 160 \\ 0.5x_1 + x_2 &\leq 90 \\ x_1, x_2 &\geq 0 \end{aligned}$$

Pregunta: ¿Cuántas unidades de cada uno producir para maximizar ganancia?

Caso 2: Refinería de petróleo

Contexto: Procesa dos tipos de crudo (A y B) para producir gas oil, lubricantes y kerosene.

Demandas mínimas diarias:

- Gas oil: 14 ton
- Lubricantes: 10 ton
- Kerosene: 8 ton

Rendimientos por tonelada:

	Crudo A	Crudo B
Gas	0.60	0.30
Lub	0.20	0.50
Ker	0.20	0.20

Costos:

- Crudo A: \$6000/ton
- Crudo B: \$8000/ton

Objetivo: Minimizar costo total cumpliendo demandas.

Variables:

- x_1: ton de crudo A
- x_2: ton de crudo B

$$\text{mín } Z = 6000x_1 + 8000x_2$$

Caso 3: Problema de riesgo moral

Contexto: Empresa diseña esquema de pagos para incentivar esfuerzo alto.

Esfuerzo alto (H):

- P(éxito) $= 0.8$
- P(fracaso) $= 0.2$
- Costo: 2

Esfuerzo bajo (L):

- P(éxito) $= 0.5$
- P(fracaso) $= 0.5$
- Costo: 0

Variables:

- w_s: pago si éxito
- w_f: pago si fracaso

Modelo:

$$\begin{aligned} \text{mín} \quad & 0.8w_s + 0.2w_f \\ \text{s.a.:} \quad & 0.3w_s - 0.3w_f \geq 2 \\ & 0.8w_s + 0.2w_f \geq 2 \\ & w_s, w_f \geq 0 \end{aligned}$$

Minimizar salario esperado incentivando esfuerzo alto

Caso 4: Asignación en call center

Contexto: Necesita cubrir horarios con dos tipos de empleados.

Datos:

- **Atención cliente:**
  - Salario: \$4000/sem
  - Eficiencia: 90 %
- **Información:**
  - Salario: \$3500/sem
  - Eficiencia: 75 %

Restricciones:

- Mínimo 50 empleados
- Al menos 20 de información
- Eficiencia promedio $\geq$ 82 %

Modelo:

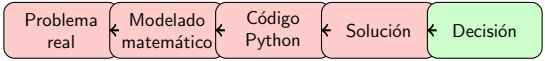
$$
\begin{aligned}
\text{mín } Z &= 4000x_1 + 3500x_2 \\
\text{s.a.:} \quad x_1 + x_2 &\geq 50 \\
x_2 &\geq 20 \\
\frac{90x_1 + 75x_2}{x_1 + x_2} &\geq 82 \\
x_1, x_2 &\geq 0
\end{aligned}
$$

Objetivo: Minimizar costo de nómina cumpliendo requisitos.

Integración

Del problema a la solución

Flujo de trabajo completo



Herramientas en cada etapa:

1. **Problema real:** Análisis y comprensión del contexto
2. **Modelado:** Identificar variables, objetivo, restricciones
3. **Código:** linprog (algebraico) o PuLP (declarativo)
4. **Solución:** Interpretación de resultados numéricos
5. **Decisión:** Aplicación práctica en la organización

Integración con Pandas

Caso real: Los datos suelen venir en archivos (CSV, Excel).

Cargar datos y formular modelo

```
import pandas as pd
from pulp import *

# Cargar datos de empleados
df = pd.read_csv('empleados.csv')

# df tiene columnas: area, salario, eficiencia
areas = df['area'].unique()

# Crear variables
x = {area: LpVariable(f"x_{area}", lowBound=0, cat='Integer')
     for area in areas}

# Modelo
modelo = LpProblem("Asignacion", LpMinimize)

# Objetivo: minimizar costo
modelo += lpSum([df[df['area']==a]['salario'].iloc[0] * x[a]
                 for a in areas])

# Restricciones...
modelo.solve()
```

Análisis de sensibilidad

¿Qué pasa si cambian los parámetros?

Preguntas típicas

- ¿Cuánto vale una hora adicional de máquina?
- ¿Vale la pena contratar un empleado más?
- ¿Qué restricción es más limitante?
- ¿Cómo afecta un cambio en los costos?

Precios sombra:

- Valor marginal de relajar una restricción
- ¿Cuánto aumenta Z si b_i crece en 1?

Holguras:

- Recursos no utilizados
- Holgura = b_i - (uso real)
- Si holgura ¿ 0, recurso sobra

El modelo no termina con la solución óptima

Buenas prácticas en PL

1. **Definir variables claramente**
   - Usar nombres descriptivos
   - Documentar unidades
2. **Verificar linealidad**
   - No productos entre variables
   - No exponentes, logaritmos
3. **Validar restricciones**
   - ¿Son realistas?
   - ¿Faltan restricciones?
4. **Interpretar resultados**
   - ¿Tienen sentido económico?
   - ¿Son implementables?
5. **Documentar el modelo**
   - Explicar variables y restricciones
   - Facilita mantenimiento y revisión

Limitaciones de la PL

Supuestos que deben cumplirse

1. **Proporcionalidad:** Efecto lineal en objetivo y restricciones
2. **Aditividad:** Recursos se suman
3. **Divisibilidad:** Variables continuas (no siempre realista)
4. **Certeza:** Parámetros conocidos con certeza

¿Qué hacer si no se cumple?

- **Variables enteras:** Programación lineal entera (ILP)
- **No linealidad:** Programación no lineal (NLP)
- **Incertidumbre:** Programación estocástica
- **Múltiples objetivos:** Programación multi-objetivo

PL es poderosa, pero no resuelve todo

Cierre

$$\text{Problema} \xrightarrow{\text{Modelar}} \text{Ecuaciones} \xrightarrow{\text{Python}} \text{Solución} \xrightarrow{\text{Analizar}} \text{Decisión}$$

Lo que aprendimos hoy:

- Los problemas organizacionales se modelan como sistemas de ecuaciones
- La programación lineal optimiza recursos limitados
- Python (linprog y PuLP) facilita la resolución
- La interpretación es tan importante como el cálculo

La optimización no reemplaza la decisión humana, la informa

Actividad propuesta

Problema: Contratación de personal
Una empresa necesita contratar empleados en Backend y Data.

Restricciones:

- Mínimo 10 empleados en total
- Al menos 3 de Backend y 3 de Data
- Suma de scores debe ser ≥ 600

Objetivo: Minimizar nómina total.

Tareas:

1. Cargar dataset de empleados (disponible en campus)
2. Calcular salarios y scores promedio por área
3. Formular modelo con PuLP
4. Resolver y analizar resultados
5. Interpretar solución: ¿cuántos contratar de cada área?

Preguntas

¿Preguntas?

"La optimización matemática es el arte de tomar las mejores decisiones posibles"

Fuentes y bibliografía

- Fraquelli, A. D. & Gache, A. L. (2019). *Notas de álgebra teórico-prácticas*. FCE-UBA.
- Hilpisch, Y. (2018). *Python for Finance: Mastering Data-Driven Finance* (2nd ed.). O'Reilly Media.
- James, G., et al. (2023). *An Introduction to Statistical Learning: with Applications in Python*. Springer.
- Documentación de SciPy: `https://docs.scipy.org/doc/scipy/reference/optimize.html`
- Documentación de PuLP: `https://coin-or.github.io/pulp/`

Materiales educativos con fines académicos.